

激光雷达SLAM建图

一、任务描述

自动驾驶车辆是实现自动驾驶的第一步，需要知道自己身处何种环境。“我在哪？”，“我要去哪？”，“如何去？”是自动驾驶需要关注的三个问题。要想清晰定义这三个问题，一张准确的地图是自动驾驶的基础。本次实训任务，需要同学在实际场地搭建地图，开启自动驾驶汽车，使用手柄遥控汽车，使用激光雷达对实际场景进行建图。通过激光雷达的数据获取，记录整个地图中的情况（哪里是障碍物，哪里是可行区域）。

二、任务目标

- 了解Gmapping算法原理。
- 掌握自动驾驶汽车的基本操作。
- 掌握自动驾驶汽车SLAM建图过程。

三、任务环境

- 操作系统：Ubuntu20.04 ROS-neotic
- 工具软件：NoMachine
- 必备硬件：自动驾驶汽车

四、任务原理

本次实训任务最终采用的是ROS中的gmapping模块来实现建图过程，gmapping算法是一种基于激光雷达的机器人定位与地图构建方法。它采用栅格地图（gridmap）来表示室内环境，利用激光雷达数据，以及机器人运动信息，来实现机器人位置估计和环境地图构建。

4.1 Gmapping算法工作流程

- （1）激光雷达传感器对周围环境进行扫描，将激光数据转换成2D栅格地图。
- （2）采用概率滤波算法（Probabilistic Filtering），使用当前激光数据和机器人的位姿信息，估计机器人的位姿。
- （3）使用激光数据和机器人位姿信息，更新栅格地图中的障碍物点和每个栅格的概率值，从而更新地图。
- （4）将构建的栅格地图，保存成文件。

4.2 SLAM问题的概率描述

首先，一个完整的SLAM问题是在给定传感器数据的情况下，同时进行机器人位姿和地图的估计的问题。然而，现实的情况是这样的，如果需要得到一个精确的位姿需要与地图进行匹配，如果需要得到一个好的地图需要有精确的位姿才能做到，显然这是一个相互矛盾的问题。

4.2.1 SLAM问题概率描述

概率描述如下式：

$$p(x_{1:t}, m | u_{1:t}, z_{1:t})$$

显然是一个条件联合概率分布，用在移动机器人从开机到t时刻一系列传感器测量数据 $z_{1:t}$ （这里当然是scan）以及一系列控制数据 $u_{1:t}$ （这里认为是odom）的条件下，同时对地图 m 、机器人轨迹状态 $x_{1:t}$ 进行的估计来描述SLAM问题。

4.2.2 SLAM问题分解

FastSLAM算法独辟蹊径，采用RBPF方法，将SLAM算法分解成两个问题。一个是机器人定位问题，另一个是已知机器人位姿进行地图构建的问题。分解过程的公式推导如下式：

$$\begin{aligned} p(x_{1:t}, m | u_{1:t}, z_{1:t}) & \xrightarrow{\text{通过条件联合概率分布化简: } p(A, B | Z) = P(A | Z)P(B | A, Z)} \\ &= p(x_{1:t} | u_{1:t}, z_{1:t}) p(m | x_{1:t}, u_{1:t}, z_{1:t}) \\ & \xrightarrow{\text{u}_{1:t} \text{与} m \text{无关, 可直接化简, 因为有轨迹+测量即可闭式计算地图}} \\ &= p(x_{1:t} | u_{1:t}, z_{1:t}) p(m | x_{1:t}, z_{1:t}) \end{aligned} \quad (3.2)$$

其中 $p(x_{1:t} | u_{1:t}, z_{1:t})$ 表示估计机器人的轨迹， $p(m | x_{1:t}, z_{1:t})$ 表示在已知机器人轨迹和传感器观测数据情况下，进行地图构建的闭式计算。这样SLAM问题就分解成两个问题。其中已知机器人位姿的地图构建是个简单问题，所以机器人位姿的估计是一个重点问题。

4.2.3 机器人轨迹增量估计分解

FastSLAM算法采用粒子滤波来估计机器人的位姿，并且为每一个粒子构建一个地图。所以，每一个粒子都包含了机器人的轨迹和对应的环境地图。现在我们着重研究一下 $p(x_{1:t} | u_{1:t}, z_{1:t})$ 估计机器人的轨迹。通过使用贝叶斯准则对 $p(x_{1:t} | u_{1:t}, z_{1:t})$ 进行公式推导如式：

$$\begin{aligned} p(x_{1:t} | u_{1:t}, z_{1:t}) &= p(x_{1:t} | z_t, u_{1:t}, z_{1:t-1}) \xrightarrow{\text{转换, 因为要寻找} z_t \text{与} x_t \text{的关系}} \\ & \xrightarrow{\text{根据条件贝叶斯公式化简}} \\ &= \eta p(z_t | x_t, u_{1:t}, z_{1:t-1}) p(x_{1:t} | u_{1:t}, z_{1:t-1}) \\ & \xrightarrow{\text{从理论上讲, } x_t \text{足以预测} z_t, z_t \text{与其他无关, 可以直接约减}} \\ &= \eta p(z_t | x_t) p(x_{1:t} | u_{1:t}, z_{1:t-1}) \\ & \xrightarrow{\text{根据条件联合概率公式化简 并且, } p(x_{1:t}) = p(x_t, x_{1:t-1})} \\ &= \eta p(z_t | x_t) p(x_t | x_{1:t-1}, u_{1:t}, z_{1:t-1}) p(x_{1:t-1} | u_{1:t}, z_{1:t-1}) \\ & \xrightarrow{\text{从理论上讲, } x_t \text{只与} x_{t-1} \text{和} u_t \text{有关, } x_t \text{与其他无关, 可以直接约减}} \\ &= \eta p(z_t | x_t) p(x_t | x_{t-1}, u_t) p(x_{1:t-1} | u_{1:t-1}, z_{1:t-1}) \end{aligned} \quad (3.3)$$

经过上面的公式推导，这里将机器人轨迹估计转化成一个增量估计的问题，用 $p(x_{1:t-1} | u_{1:t-1}, z_{1:t-1})$ 表示上一时刻的机器人轨迹，用上一时刻的粒子群表示。每一个粒子都用运动学模型 $p(x_t | x_{t-1}, u_t)$ 进行状态传播，这样就得到每个粒子对应的预测轨迹。对于每一个传播之后的粒子，用观测模型 $p(z_t | x_t)$ 进行权重计算归一化处理，这样就得到该时刻的机器人轨迹。之后根据估计的轨迹以及观测数据进行地图构建。

推导公式参考：

$$p(x|y, z) = \frac{p(y|x, z)p(x|z)}{p(y|z)} = \eta p(y|x, z)p(x|z) \quad \text{条件贝叶斯公式}$$

$$p(x, y|z) = p(x|y, z)p(y|z) \quad \text{条件联合概率公式}$$

4.3 Gmapping算法

从上一节的描述上面我们已经知道SLAM问题被分解成了两个问题，机器人轨迹估计问题和已知机器人轨迹的地图构建问题。FastSLAM算法中的机器人轨迹估计问题使用的粒子滤波方法。由于使用的是粒子滤波，将不可避免的带来两个问题。第一个问题，当环境大或者机器人里程计误差大时，需要更多的粒子才能得到较好的估计，这时将造成内存爆炸；第二个问题，粒子滤波避免不了使用重采样，以确保当前粒子的有效性，然而重采样带来的问题就是粒子耗散，粒子多样性的丢失。由于这两个问题出现，导致FastSLAM算法理论上可行实际上却不能实现。针对以上问题gmapping提出了两种针对性的解决方法。也就是说gmapping是基于FastSLAM算法将RBPF方法变成了现实。

4.3.1 降低粒子数量

问题由来：每一个粒子都包含自己的栅格地图，对于一个稍微大一点的环境来说，每一个粒子都会占用比较大的内存。如果机器人里程计的误差比较大，即proposal分布（提议分布）跟实际分布相差较大，则需要较多的粒子才能比较好的表示机器人位姿的估计，这样将会造成内存爆炸。

目的：通过降低粒子数量的方法大幅度缓解内存爆炸。

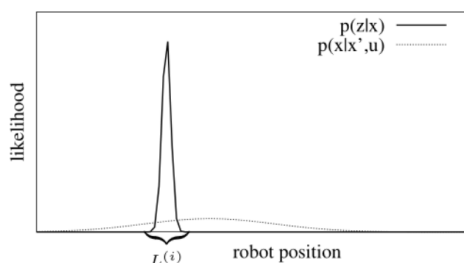
分析：里程计的概率模型比较平滑，是一个比较大的范围，如果对整个范围采样将需要很多的粒子，如果能找到一个位姿优值，在其周围进行小范围采样，这样不就可以降低粒子数量了嘛。

方法一：直接采用极大似然估计的方式，根据粒子的位姿的预测分布和与地图的匹配程度，通过扫描匹配找到粒子的最优位姿参数，就用该位姿参数，直接当做新粒子的位姿。如下图所示，从普通的proposal分布采样替换为用极大似然估计提升采样的质量。这也就是gmapping算法中的做法。

$$x_t^i \sim p(x_t | u_t, x_{t-1}^i) \quad \longrightarrow \quad x_t^i = \arg \max_{x_t} \{p(z_t | x_t, m) p(x_t | u_t, x_{t-1}^i)\}$$

方法二：gmapping算法通过最近一帧的观测（scan），把proposal分布限制在一个狭小的有效区域。然后在正常的对proposal分布进行采样。该方法是gmapping论文中详细描述的方法。

• 假设：



激光雷达的匹配比里程计的测量精确很多，从分布上来说，激光雷达匹配的方差要比里程计模型的方差小很多

如果proposal分布用激光匹配来表示，则可以把采样范围限制在一个比较小的区域，因此可以用更少的粒子即覆盖了机器人位姿的概率分布。也就是说将proposal分布从里程计的观测模型变换到了激光雷达观测模型。具体公式推导如下所示：

用观测模型为预测模型限制一个密集区域代替预测分布
预测分布也叫prosoal分布 $p(x_t | x_{t-1}, u_t) \rightarrow p(x_t | x_{t-1}, u_t, z_t, m)$ 加入地图m和观测z_t

$$p(x_t | x_{t-1}, u_t, z_t, m) = p(x_t | z_t, u_t, x_{t-1}, m) \quad \longrightarrow \quad \text{替换}$$

$$\downarrow \text{根据条件贝叶斯公式化简}$$
$$= \eta p(z_t | x_t, u_t, x_{t-1}, m) p(x_t | x_{t-1}, u_t, m)$$

$$\downarrow \text{将无关项约减掉，简化公式}$$
$$= \eta p(z_t | x_t, m) p(x_t | x_{t-1}, u_t)$$

$$\downarrow \text{由于} x_t \text{属于尖峰区域，预测分为可视为常数化简}$$
$$= \eta p(z_t | x_t, m), x_t \in L^{(i)}$$

从上图可以看到激光雷达观测模型方差较小，假设其服从高斯分布，使用多元正态分布公式即可进行计算，于是求解高斯分布是下面需要做的。

同方法一，首先通过极大似然估计找到局部极值 x_t ，认为该局部极值距离高斯分布的均值比较近，于是在位姿 x_t 附近取了 k 个位姿；对 k 个位姿进行打分（位姿匹配），这样我们就具有 k 个位姿，以及它们与地图匹配的得分；我们又假设它们服从高斯分布，计算出它们的均值和方差之后，即可使用多元正态分布，对机器人位姿的估计进行概率计算。矢量均值和协方差的计算。具体如下式(3.4) (3.5)

$$\text{矢量均值} \quad \mu = \frac{1}{\eta} \sum_{j=1}^K x_j p(z_t | x_j, m) \quad (3.4)$$

$$\text{协方差} \quad \Sigma = \frac{1}{\eta} \sum_{j=1}^K (x_j - \mu)(x_j - \mu)^T p(z_t | x_j, m) \quad (3.5)$$

多元正态分布公式如下，可以根据均值和协方差，直接计算得出后验概率。

$$p(x) = \det(2\pi\Sigma)^{-\frac{1}{2}} \exp\left\{-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right\}$$

当然，这里不仅要计算每一个粒子的轨迹的后验概率，还要为每一个粒子计算权重，这里对权重具体的计算方式就不做描述了。论文里有。

4.3.2 缓解粒子耗散

问题由来：gmapping算法采用粒子滤波算法对移动机器人轨迹进行估计，必然少不了粒子重采样的过程。随着采样次数的增多，会出现所有粒子都从一个粒子复制而来，这样粒子的多样性就完全丧失了。

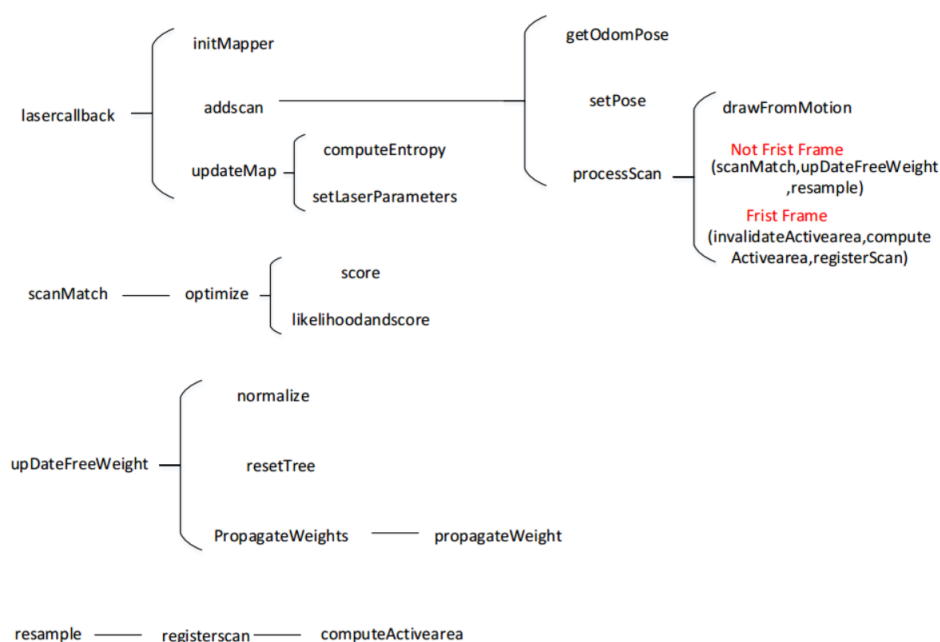
目的：缓解粒子耗散

分析：gmapping算法从算法原理上使用粒子滤波，不可避免的要进行粒子重采样，所以，问题实实在在的存在，不能解决，只能从减少重采样的思路走。

方法：gmapping提出选择性重采样的方法，根据所有粒子自身权重的离散程度（也就是权重方差）来决定是否进行粒子重采样的操作。于是就出现了式（3.7），当 N_{eff} 小于某个阈值，说明粒子差异性过大，进行重采样，否则，不进行。

$$N_{eff} = \frac{1}{\sum (w^i)^2} \quad (3.7)$$

4.4 算法函数的调用流程



五、任务步骤

步骤一、构建地图

构建真实地图，需要注意地图的尺寸，地图尺寸在ros_space/map_size.yaml中定义。单位为“米”。

```
1 map_x : 3.6  
2 map_y : 3.6
```

真实地图场景如下：



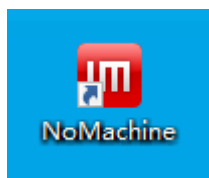
步骤二、打开自动驾驶汽车

打开自动驾驶汽车，首先按下总开关，然后开启英特尔NUC电源。

步骤三、远程连接

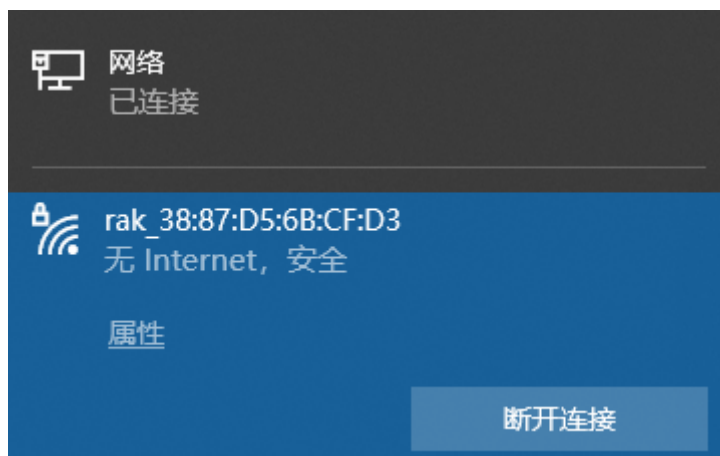
2.1 安装NoMachine

在windows上安装远程连接客户端NoMachine：



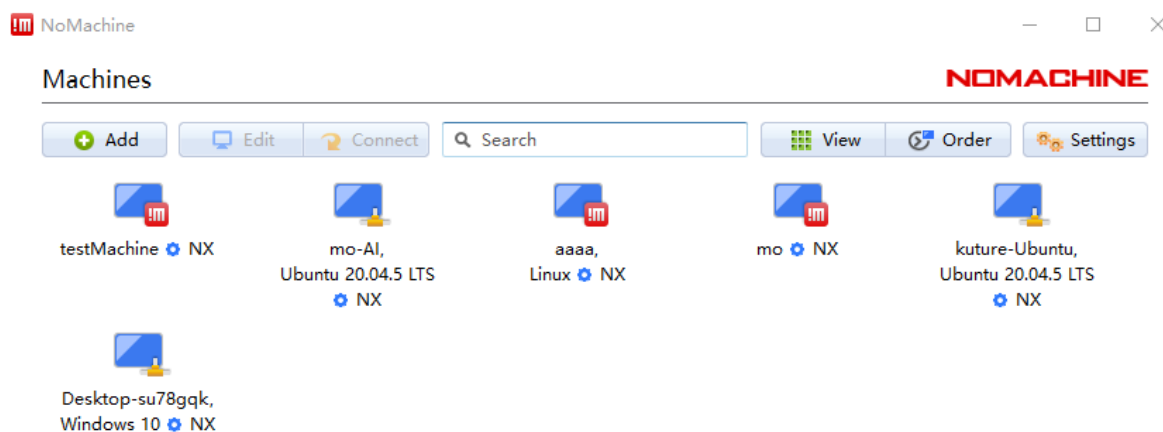
2.2 连接wifi

找到自动驾驶汽车WiFi，连接汽车wifi，密码为123456：

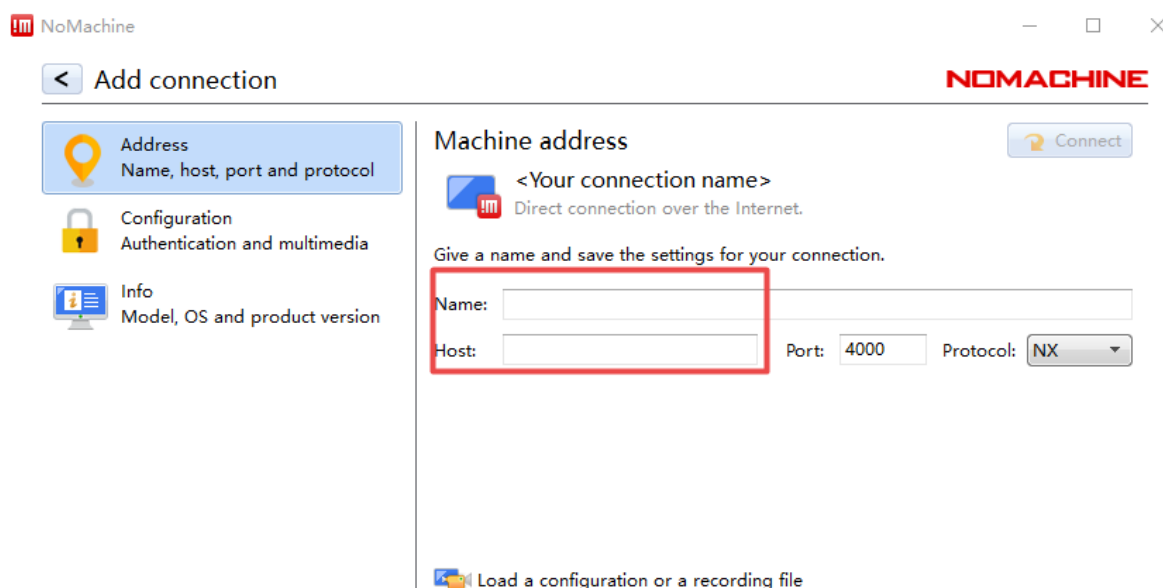


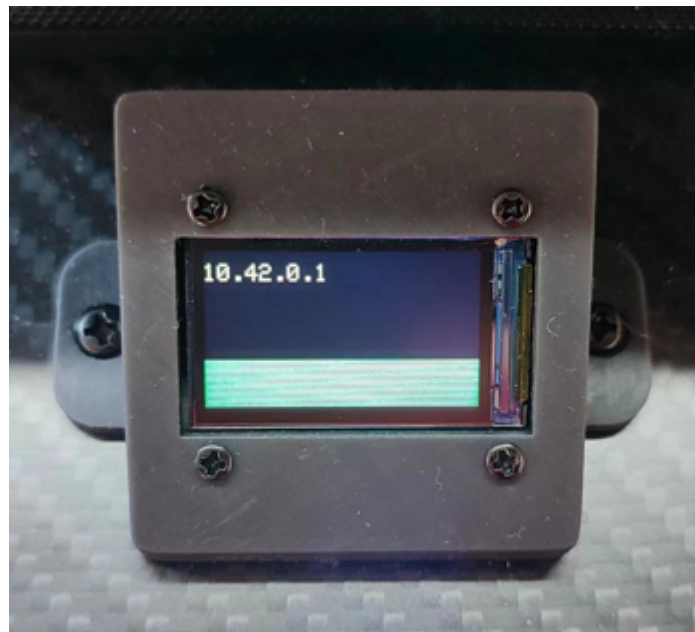
2.3 添加自动驾驶汽车

双击打开NoMachine, 添加自动驾驶汽车:

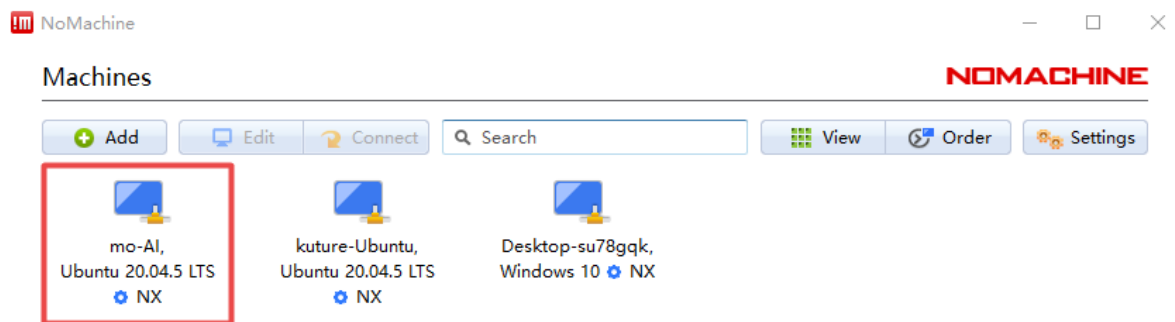


点击“Add”后, 跳转到添加页面, “Name”栏可以自己命名, Host填写小车屏幕上的ip即可。





在NoMachine主界面可以看到添加的自动驾驶汽车：



2.4 连接自动驾驶汽车

双击添加好的自动驾驶汽车，输入用户名密码，用户名：mo，密码：123456，进入到小车系统界面：

Please type your username and password to login.



Username mo

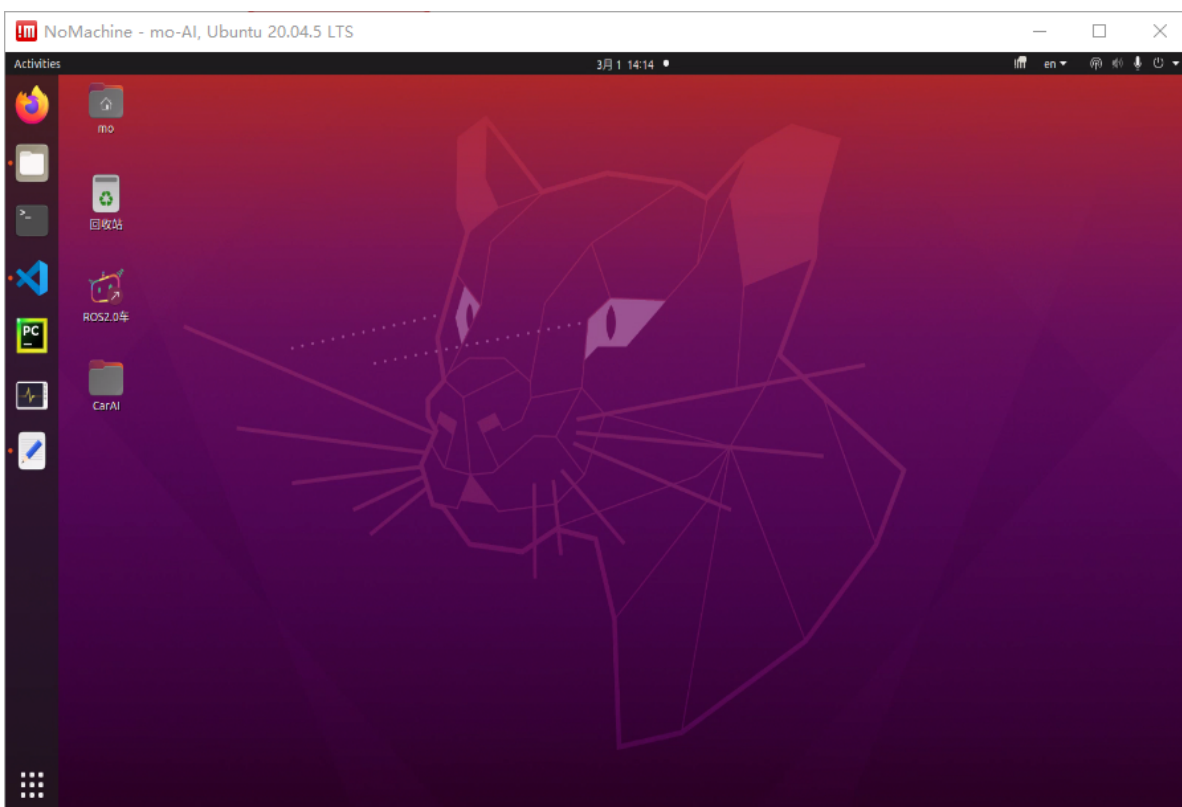
Password

☒ Save this password in the connection file

Authentication failed, please try again.

Cancel

Login



步骤四、开启建图功能

打开桌面自动驾驶汽车桌面管理软件“ROS2.0车”，选择“机器人-LAB”→“激光雷达导航”

自动驾驶-LAB

机器人-LAB

场景体验

设置



激光雷达导航

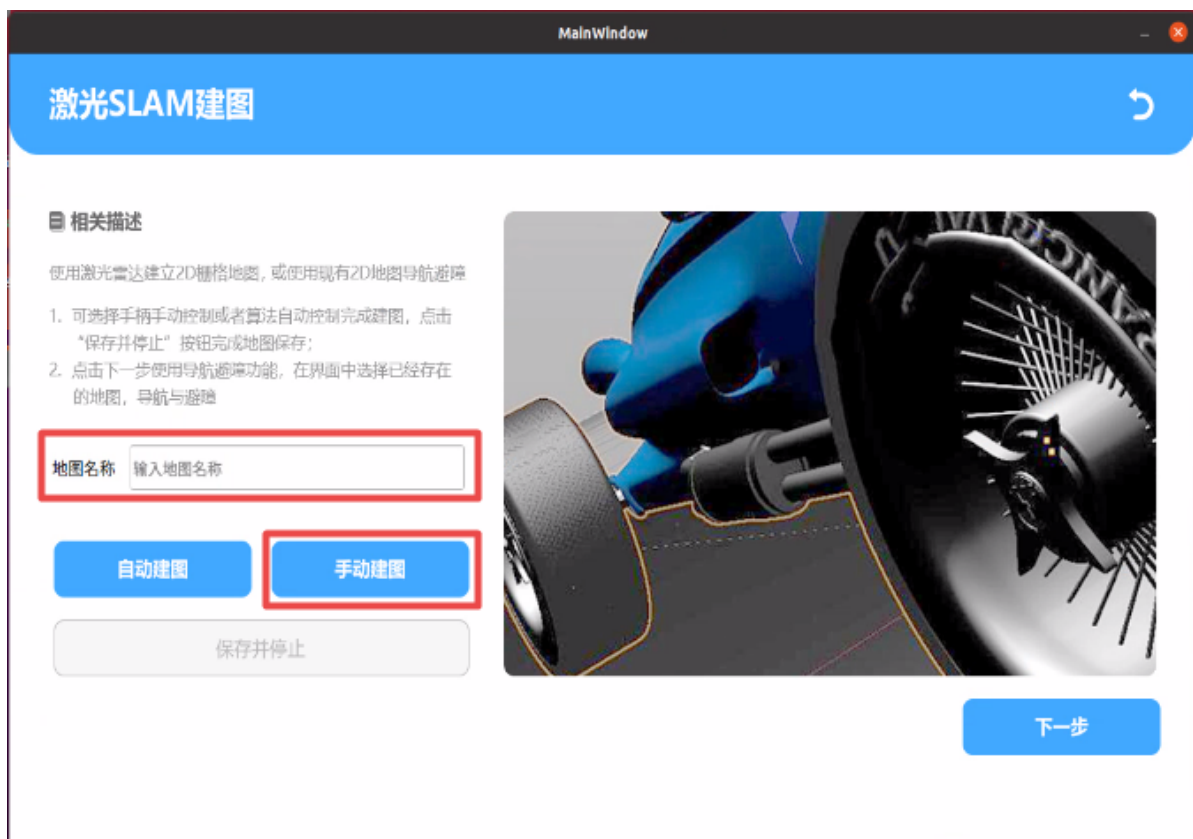


深度视觉导航

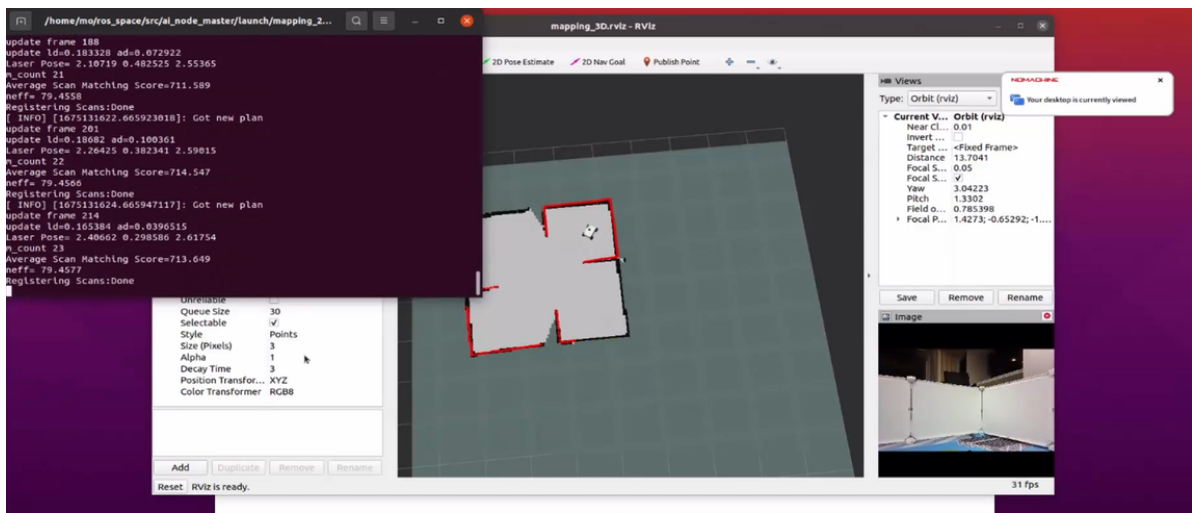


仿真环境

进入建图界面：



输入地图名字，点击“手动建图”，打开RVIZ界面，右下角为摄像头视角。



步骤五、遥控自动驾驶汽车，建图并保存

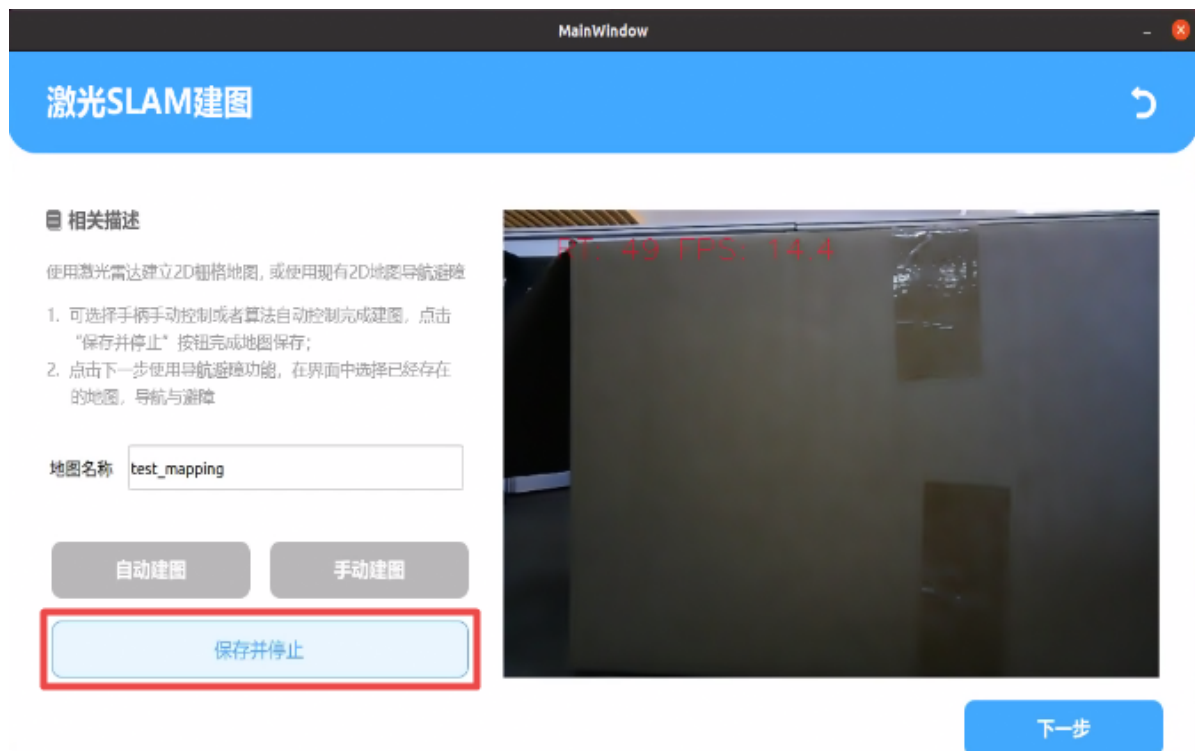
长按手柄启动按键，与小车上的无线接收器进行配对。



通过手柄上的两个摇杆，控制小车的前后左右移动：



建图结束后，点击“保存并停止”，停止建图并保存创建好的地图：



创建好的地图保存在“/home/mo/ros_space/src/ai_node_master/maps/”中



步骤六 代码流程编写

1. 创建文件"/home/mo/ros_space/src/ai_node_master/launch/mapping_2d.launch"，添加如下内容，此文件体现了2d建图用到的节点信息：

```
<launch>
  <!--param参数配置-->
  <param name="node_id_closer" value="/mapping_2d" />
  <node pkg="ai_node_master" type="ros_closer" name="mapping_2d"
output="screen" required="true" />

  <!-- 2D建图节点 -->
  <include file="$(find
navigation_settings)launch/robot_gmapping.launch.xml"/>
  <!-- <include file="$(find cartographer_local)launch/demo_revo_lds.launch"/>
  -->

  <!-- 激光雷达驱动 -->
  <include file="$(find rplidar_ros)/launch/rplidar.launch" />

  <!-- 底盘与深度摄像头API -->
  <include file="$(find ai_node_master)/cfg/API_cfg.launch" >
  </include>

  <!-- 手柄驱动 -->
  <rosparam file="/home/mo/ros_space/joystick.yaml" command="load" />
  <node pkg="joy" type="joy_node" name="joy_node" output="screen"
respawn="true"/>
  <node pkg="robot_driver" type="joystick_controller"
name="joystick_controller" output="screen" respawn="true"/>

  <!-- 可视化界面 -->
  <node pkg="rviz" type="rviz" name="rviz" args="-d $(find
urdf_car)/rviz/mapping_2D.rviz" output="screen"/>

</launch>
```

代码中设置建图节点的语句为 `<include file="$(find navigation_settings)launch/robot_gmapping.launch.xml"/>`。

2. 创建文

件"/home/mo/ros_space/src/navigation_settings/launch/robot_gmapping.launch.xml"，添加如下代码，此代码调用ROS中的gmapping模块：

```
<launch>
  <arg name="scan_topic" default="scan" />
  <arg name="base_frame" default="base_footprint"/>
  <arg name="odom_frame" default="odom"/>

  <node pkg="gmapping" type="slam_gmapping" name="slam_gmapping"
output="screen"> <!--此处调用ROS的gmapping模块-->
    <param name="base_frame" value="$(arg base_frame)"/> <!--底盘坐标系-->
    <param name="odom_frame" value="$(arg odom_frame)"/> <!--里程计坐标系-->
    <param name="map_update_interval" value="0.1"/> <!--更新时间(s)，每
多久更新一次地图，不是频率-->
```

```

    <param name="maxUrange" value="6.0"/>                                <!--激光雷达最大可用
距离，在此之外的数据截断不用-->
    <param name="maxRange" value="5.5"/>                                <!--激光雷达最大距离-
->
    <param name="sigma" value="0.05"/>
    <param name="kernelSize" value="1"/>
    <param name="lstep" value="0.15"/>
    <param name="astep" value="0.1"/>
    <param name="iterations" value="5"/>
    <param name="lsigma" value="0.075"/>
    <param name="ogain" value="3.0"/>
    <param name="lskip" value="0"/>
    <param name="srr" value="0.01"/>
    <param name="srt" value="0.02"/>
    <param name="str" value="0.01"/>
    <param name="stt" value="0.02"/>
    <param name="linearUpdate" value="0.2"/>
    <param name="angularUpdate" value="0.1"/>
    <param name="temporalUpdate" value="2.0"/>
    <param name="resampleThreshold" value="0.5"/>
    <param name="particles" value="80"/>
    <param name="minimumScore" value="30"/>
    <param name="xmin" value="-5.0"/>
    <param name="ymin" value="-5.0"/>
    <param name="xmax" value="5.0"/>
    <param name="ymax" value="5.0"/>

    <param name="delta" value="0.05"/>                                <!--栅格大小-->
    <param name="llsamplerange" value="0.01"/>
    <param name="llsamplestep" value="0.01"/>
    <param name="lasamplerange" value="0.005"/>
    <param name="lasamplestep" value="0.005"/>
    <remap from="scan" to="$(arg scan_topic)"/>
  </node>
</launch>

```

同学可以自行修改以上文件参数，观察不同的参数对与自动驾驶汽车的运行效果，有何不同的作用。

3. 创建文件"/home/mo/ros_space/src/rplidar_ros-master/launch/rplidar.launch"，添加如下代码，此代码调用激光雷达驱动模块：

```

<launch>
  <!--此处调用rplidarNode模块-->
  <node name="rplidarNode"          pkg="rplidar_ros"  type="rplidarNode"
output="screen" respawn="true">
    <param name="serial_port"        type="string" value="/dev/RF_lidar"/>
    <param name="serial_baudrate"    type="int"     value="115200"/><!--A1/A2
-->
    <!--param name="serial_baudrate"    type="int"     value="256000"--><!--A3
-->
    <param name="frame_id"           type="string" value="radar"/>
    <param name="inverted"           type="bool"   value="false"/>
    <param name="angle_compensate"    type="bool"   value="true"/>
  </node>
</launch>

```

4. 创建文件"/home/mo/ros_space/src/ai_node_master/cfg/API_cfg.launch"，添加如下代码，此代码调用底盘驱动模块：

```
<launch>
  <!--参数配置-->
  <arg name="load_depth_driver"    default="false" />    <!--读取深度硬件-->
  >
  <arg name="load_rgb_driver"      default="false" />    <!--读取RGB摄像头
硬件-->
  <arg name="encoder_only"        default="false" />    <!--禁用激光雷达矫正-->
  <arg name="driver_node"         default="true" />     <!--开启ROS自带底
盘节点控制-->

  <arg name="use_API"             default="true" />     <!--使用API-->
  <arg name="IMU_msg"             default="false" />    <!--API获取IMU信
息-->
  <arg name="driver_msg"          default="false" />    <!--API获取底盘信
息-->
  <arg name="msg_debug"           default="false" />    <!--打印debug-->

  <!--API读取节点-->
  <node if="$(arg use_API)" name="ros_communicator" pkg="ai_node_master"
type="ros_communicator.py" output="screen" >
    <param name="depth_stream_flag" value="$(arg load_depth_driver)" />
    <param name="rgb_stream_flag"   value="$(arg load_rgb_driver)" />
    <param name="IMU_msg"           value="$(arg IMU_msg)" />
    <param name="driver_msg"        value="$(arg driver_msg)" />
    <param name="msg_debug"         value="$(arg msg_debug)" />
  </node>

  <!--底盘控制节点-->
  <include file="$(find ai_node_master)/launch/base_driver.launch" >
    <arg name="use_API"             value="$(arg use_API)" />
    <arg name="driver_node"         value="$(arg driver_node)" />
    <arg name="encoder_only"        value="$(arg encoder_only)" />
  </include>

  <!--深度摄像头节点-->
  <include file="$(find astra_camera)/launch/gemini.launch" >
    <arg name="load_depth_driver"   default="$(arg load_depth_driver)"
  />
    <arg name="load_rgb_driver"     default="$(arg load_rgb_driver)" />
  </include>

</launch>
```

六、任务总结

1. Gmapping是基于滤波SLAM框架的常用开源SLAM算法。
2. Gmapping基于RBpf粒子滤波算法，即将定位和建图过程分离，先进行定位再进行建图。

3. Gmapping在RBpf算法上做了两个主要的改进：改进提议分布和选择性重采样。

4. Gmapping算法优缺点：

优点： Gmapping可以实时构建室内地图，在构建小场景地图所需的计算量较小且精度较高。相比Hector SLAM对激光雷达频率要求低、鲁棒性高（Hector 在机器人快速转向时很容易发生错误匹配，建出的地图发生错位，原因主要是优化算法容易陷入局部最小值）；而相比Cartographer在构建小场景地图时，Gmapping不需要太多的粒子并且没有回环检测因此计算量小于Cartographer而精度并没有差太多。Gmapping有效利用了车轮里程计信息，这也是Gmapping对激光雷达频率要求低的原因：里程计可以提供机器人的位姿先验。而Hector和Cartographer的设计初衷不是为了解决平面移动机器人定位和建图，Hector主要用于救灾等地面不平坦的情况，因此无法使用里程计。而Cartographer是用于手持激光雷达完成SLAM过程，也就没有里程计可以用。

缺点： 随着场景增大所需的粒子增加，因为每个粒子都携带一幅地图，因此在构建大地图时所需内存和计算量都会增加。因此不适合构建大场景地图。并且没有回环检测，因此在回环闭合时可能会造成地图错位，虽然增加粒子数目可以使地图闭合但是以增加计算量和内存为代价。所以不能像Cartographer那样构建大的地图，虽然论文生成几万平米的地图，但实际我们使用中建的地图没有几千平米时就会发生错误。Gmapping和Cartographer一个是基于滤波框架SLAM另一个是基于优化框架的SLAM，两种算法都涉及到时间复杂度和空间复杂度的权衡。Gmapping牺牲空间复杂度保证时间复杂度，这就造成Gmapping不适合构建大场景地图，试想一下你要构建200乘200米的环境地图，栅格分辨率选择5厘米，每个栅格占用一字节内存，那么一个粒子携带的地图就需要16M内存，如果是100个粒子就需要1.6G内存。如果地图变成500乘500米，粒子数为200个，运算量将会很大。翻看Cartographer算法，优化相当于地图中只用一个粒子，因此存储空间比较Gmapping会小很多倍，但计算量大。优化图需要复杂的矩阵运算，这也是谷歌为什么还有弄个ceres库出来的原因。